

Coding Challenge: Wordle-Lite Guesser

Objective:

In this assignment, you will build a text-based, simplified version of the popular game Wordle in V (Vlang). This challenge is designed to help you practice core V programming concepts up to arrays and strings, including string indexing, byte-to-string conversion, array operations, and custom control flow with loop and match structures.

Game Rules & Specification:

- **Secret Word:** The player must guess a pre-defined 5-letter word (e.g., "Vlang").
- **Attempts Limit:** The player has exactly 5 attempts to guess the secret word.
- **Validation:** If a guess is not exactly 5 letters long, print a warning and let the player try again without losing an attempt.
- **Feedback Markers:** For every incorrect guess, display the following feedback for each position:
 - [C] (Uppercase character in brackets) if the letter is correct and in the correct spot.
 - c? (Lowercase character with a question mark) if the letter is in the word but in the wrong spot.
 - x (Lowercase letter x) if the letter is not in the secret word at all.
- **Victory:** If the player guesses the word correctly, display a victory message and terminate immediately.
- **Game Over:** If the player runs out of guesses, display a Game Over message and reveal the secret word.

Step-by-Step Implementation Guide:

Step 1: Import Console Module

Import the standard ``os`` module at the top of your file. V's standard library uses ``os.input()`` to read text input from the terminal.

```
import os
```

Step 2: Main Entry Point & Game Setup

Declare your ``fn main()``. Initialize the target word ``secret_word`` (immutable string) and a mutable variable ``attempts`` set to 5.

```
fn main() {  
  secret_word := 'Vlang'  
  mut attempts := 5  
}
```

Step 3: Construct the Game Loop

Create a ``for attempts > 0`` loop. All guess validations and processing logic will execute inside this loop block.

```
for attempts > 0 {  
  // Loop body goes here...  
}
```

Step 4: Fetch & Validate Guess Input

Prompt the player to enter a 5-letter word. If the length is not equal to 5, print a warning and use `continue` to start the next iteration immediately without consuming an attempt.

```

guess := os.input('Attempt ${6 - attempts}/5 > ')
if guess.len != 5 {
    println('[Warning] Please enter exactly 5 characters!')
    continue
}

```

Step 5: Handle Direct Match (Victory Condition)

Convert both the player's guess and the secret word to lowercase using `.to\_lower()` and check for equality. If they are equal, output the success text and terminate the program using `return`.

```

if guess.to_lower() == secret_word.to_lower() {
    println('[Success] Correct! You guessed the word: ${secret_word}!')
    return
}

```

Step 6: Process Incorrect Guess (Setup Feedback list)

Create a mutable array `feedback` to collect markers. Run a loop from `0` to `4` (using `for i in 0 .. 5`) to check each letter position.

```

mut feedback := []string{}
for i in 0 .. 5 {
    // Inspect character at index i...
}

```

Step 7: Handle Byte-to-String Conversion

V strings are UTF-8 byte arrays under the hood. Indexing `guess[i]` returns a byte (`u8`). You must call `.ascii\_str()` to convert them to strings before performing string-matching operations.

```

guess_char := guess[i].ascii_str()
secret_char := secret_word[i].ascii_str()

```

Step 8: Evaluate Letter Match Logic

Check the match rules for each index: if it matches the same index in the secret word, append `[C]` (uppercase in brackets); if it exists elsewhere in the word, append `c?` (using `.contains()`); otherwise, append `x`.

```

if guess_char.to_lower() == secret_char.to_lower() {
    feedback << '[$ {guess_char.to_upper()}]' // Correct position
} else if secret_word.to_lower().contains(guess_char.to_lower()) {
    feedback << '${guess_char.to_lower()}?' // Wrong position
} else {
    feedback << 'x' // Not in word
}

```

Step 9: Display Output & Decrement Attempts

Outside the character loop (but inside the main game loop), join the markers using `feedback.join(" ")` and print it. Decrement the `attempts` variable.

```

println('Feedback: ${feedback.join(" ")}\n')
attempts--

```

Starter Code Template:

```
import os

fn main() {
  secret_word := 'Vlang' // 5-letter secret word
  mut attempts := 5

  println('=== Wordle-Lite Guesser ===')
  println('Guess the 5-letter word! You have ${attempts} attempts.')

  for attempts > 0 {
    guess := os.input('Attempt ${6 - attempts}/5 > ')

    if guess.len != 5 {
      println('[Warning] Please enter exactly 5 characters!')
      continue
    }

    if guess.to_lower() == secret_word.to_lower() {
      println('[Success] Correct! You guessed the word: ${secret_word}!')
      return
    }

    // TODO: 1. Initialize empty mutable feedback array
    // TODO: 2. Loop through each character position (0 to 4)
    // TODO: 3. Convert bytes to ASCII strings using .ascii_str()
    // TODO: 4. Check position match, character existence, or failure
    // TODO: 5. Print feedback, decrement attempts

    println('Feedback: [Not yet implemented]\n')
    attempts-- // Remove when implementation is complete
  }

  println('[Game Over] The secret word was: ${secret_word}')
}
```